Managing the Machine

Reviewing and Debugging AI Code



Gemini Notebook

Meet your new engineering intern.

Expectation



Not a Senior Architect.

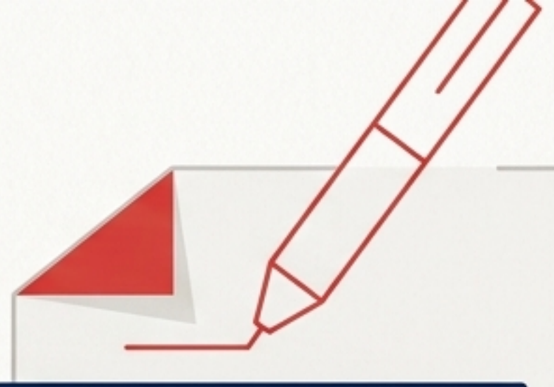
Reality



Your job is not to write the code. Your job is to manage the intern, review their drafts, and keep their chaos contained.

Gemini Notebook

You are no longer a creator.
You are an auditor.



	The Traditional Engineer	**The AI-Native Auditor** ☑
Core Action	Writes line-by-line	**Generates in blocks, verifies aggressively**
Default Trust Level	Trusts their own logic	**Trusts nothing; assumes the AI is hallucinating**
Pacing	Builds slowly, tests at the end	**Builds instantly, tests every single step**
Skill Required	Syntax memorization	**System logic and boundary setting**

Gemini Notebook

The 3-Step Audit Rule keeps chaos off your desk.

1



Check File Scope

Did the AI touch files it wasn't supposed to?

2



Verify Facts & APIs

Did the AI hallucinate tools or libraries that don't exist?

3



Enforce Simplicity

Did the AI write a tangled mess to solve a simple problem?

If a patch fails any of these three steps, it goes back to the intern. You do not fix it yourself.

Gemini Notebook

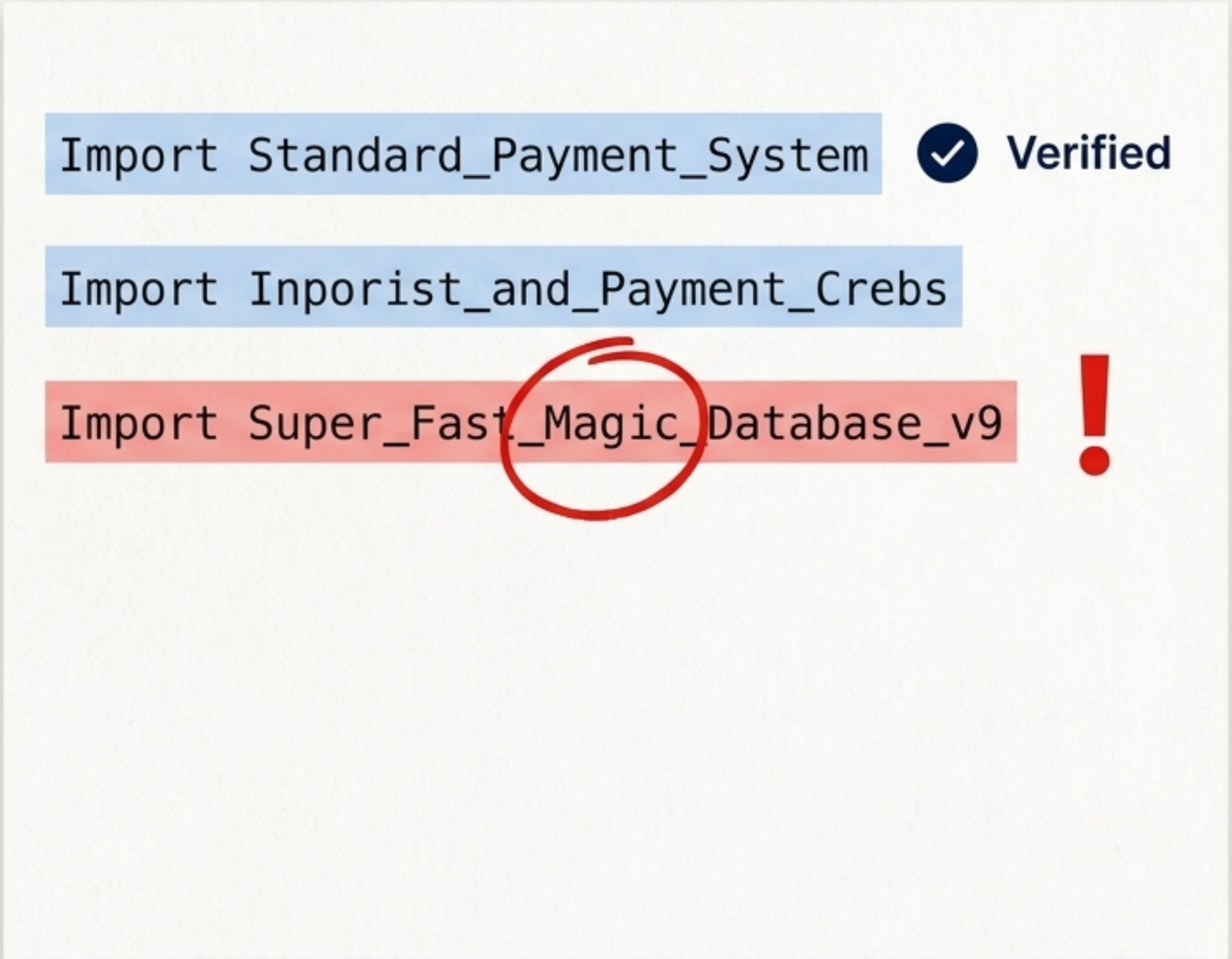
Step 1: Check the file scope before reading a single line of code.



If you asked the AI to fix a login button, and it starts modifying your core database rules... **STOP. Reject the patch immediately. Do not try to read it.**

Gemini Notebook

Step 2: Treat new code libraries like suspicious bibliography references.



The Trap

AI models want to please you. If they don't know the right tool (API) to connect to, they will seamlessly invent one that sounds real.

The Audit

Anytime the AI introduces a new library, integration, or tool you didn't explicitly ask for, copy the name and search for it online. If the documentation doesn't exist, the AI hallucinated it.

Gemini Notebook

Step 3: Reject hidden complexity and demand the thinnest working path.



AI Logic: Over-engineered edge cases and unnecessary wrappers

The AI will naturally write overly complex code to cover edge cases you don't even have yet. **Your job is to force it back to the simplest possible route**. If you can't explain the logic in plain English, the code is too complex.

Gemini Notebook

How to spot a panic patch before it breaks your build.

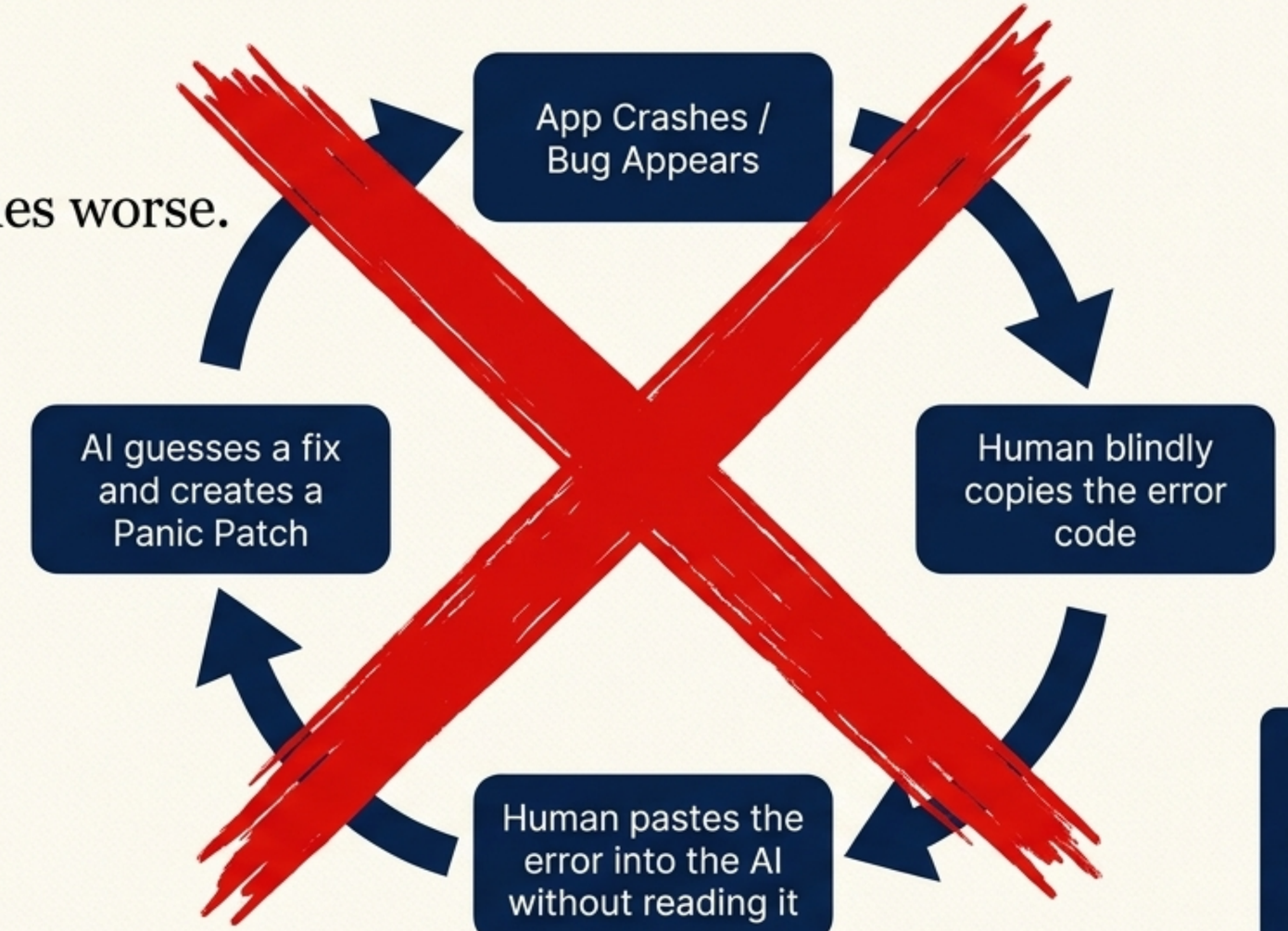
The Good Patch	The Panic Patch
✓ Touches only 1 or 2 files.	✗ Touches 5+ files simultaneously.
✓ Deletes more code than it adds.	✗ Imports brand new, mysterious libraries.
✓ Explains exactly why the previous logic failed.	✗ Adds massive blocks of complex if/else logic.
✓ Uses your existing tools and architecture.	✗ Apologizes profusely and completely rewrites the entire function.

A Panic Patch is the AI desperately guessing. Never merge a guess.

Gemini Notebook

The fastest way to ruin a project is blindly prompting through a bug.

App crashes worse.



The AI lacks context.

If you feed it raw panic, it will feed you chaotic code.

You must break this loop.

Gemini Notebook

Repro-First Debugging forces you to steer the AI, not just react to it.



Step 1: STOP

Do not copy-paste the error. **Do not open the AI chat**.



Step 2: Recreate Manually

Click through your app yourself. Can you make the error happen twice?



Step 3: Identify the Trigger

Find the exact moment it breaks (*e.g., 'It crashes when a user without a profile picture logs in'*).

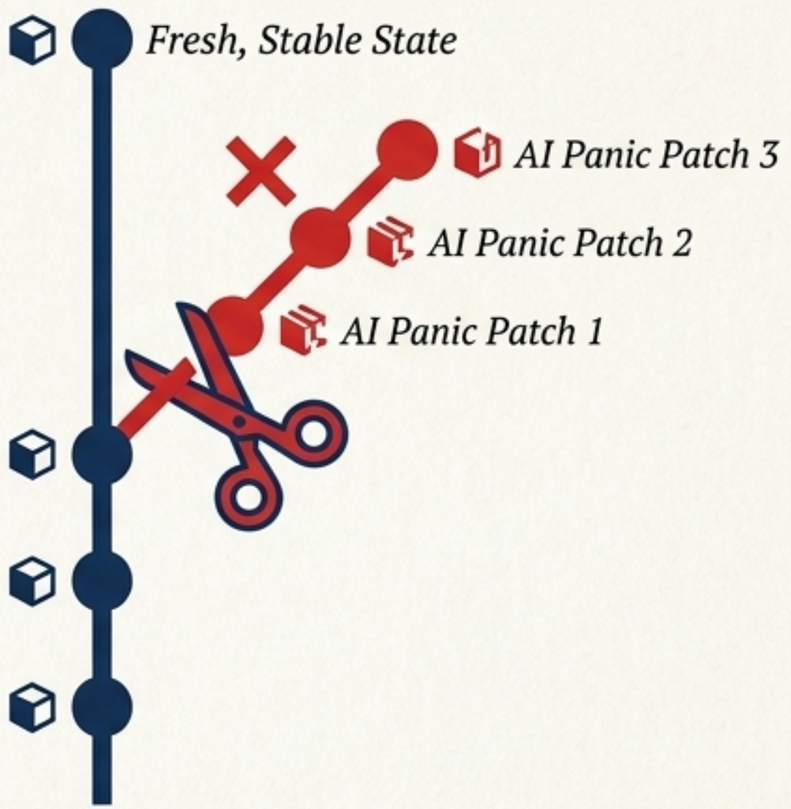


Step 4: Prompt with Context

Tell the AI exactly what you did, what you expected, and what actually happened. Now, the an AI is **solving a specific problem, not guessing**.

Gemini Notebook

Revert Discipline means knowing exactly when to use the scissors.



"Never try to fix a bad AI fix. If the code is broken and the first patch doesn't work, cut the branch. Revert back to your last working state and start over. Prune dead branches; don't tape dead leaves together."

Gemini Notebook

Sometimes the bug isn’t in the code. It’s in your instructions.

The Symptom



The AI keeps failing the 3-Step Audit, hallucinating APIs, and generating tangled knots of logic.

The Cure



Reject the patch entirely. Open your one-page spec.

Diagnostic Checklist

- ✓ Did I ask for a feature that contradicts my core data model?
- ✓ Am I trying to skip a step in the user journey?
- ✓ Is this feature actually necessary for the “aha moment”?



When the intern is consistently failing, rewrite the spec. Better instructions yield better code.

Gemini Notebook

The Solo Engineer’s Shield: You supply the discipline, the AI supplies the velocity.

Provides **infinite velocity**.

Generates the **raw materials (code blocks)**.

Tireless, but **careless**.



Provides the **steering** (**The Spec** & **Repro-First Debugging**).

Provides the **brakes** (The **3-Step Audit** & **Revert Discipline**).

You do not need to be smarter than the AI to build software.
You just need to be *more disciplined* than it.

Gemini Notebook